

中级 SQL

大纲

- 连接表达式 (join)
- 视图 (view)
- 事务 (transactions)
- 完整性约束
- SQL数据类型和模式
- SQL中的索引定义 (index)
- 授权 (authorization)

连接关系 (join)

- 连接(join)运算采用两个关系并返回另一个关系。
- 连接操作是一种笛卡尔积运算，要求两个关系中的元组匹配（在某些条件下）。它还指定了连接结果中存在的属性。
- 连接操作通常用作from子句中的子查询表达式
- 三种类型的连接：
 - 自然连接 (Natural join)
 - 内连接 (Inner join)
 - 外连接 (Outer join)

SQL中的自然连接 (Natural Join)

- 自然连接将元组的所有公共属性都匹配为相同的值，并且只保留每个公共列的一个副本。
- 例如，列出教师姓名以及他们教授的课程ID

```
select name, course_id  
from student, takes  
where student.ID = takes.ID;
```

- 使用“自然连接”结构的 SQL 中的相同查询

```
select name, course_id  
from student natural join takes;
```

SQL 中的自然连接 (续)

- from子句可以**使用**自然连接来组合多个关系：

```
select A1, A2, ... An  
from r1 natural join r2 natural join .. natural join rn  
where P;
```

学生(student) 关系

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>tot_cred</i>
00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
19991	Brandt	History	80
23121	Chavez	Finance	110
44553	Peltier	Physics	56
45678	Levy	Physics	46
54321	Williams	Comp. Sci.	54
55739	Sanchez	Music	38
70557	Snow	Physics	0
76543	Brown	Comp. Sci.	58
76653	Aoi	Elec. Eng.	60
98765	Bourikas	Elec. Eng.	98
98988	Tanaka	Biology	120

选课 (takes) 关系

<i>ID</i>	<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>	<i>grade</i>
00128	CS-101	1	Fall	2017	A
00128	CS-347	1	Fall	2017	A-
12345	CS-101	1	Fall	2017	C
12345	CS-190	2	Spring	2017	A
12345	CS-315	1	Spring	2018	A
12345	CS-347	1	Fall	2017	A
19991	HIS-351	1	Spring	2018	B
23121	FIN-201	1	Spring	2018	C+
44553	PHY-101	1	Fall	2017	B-
45678	CS-101	1	Fall	2017	F
45678	CS-101	1	Spring	2018	B+
45678	CS-319	1	Spring	2018	B
54321	CS-101	1	Fall	2017	A-
54321	CS-190	2	Spring	2017	B+
55739	MU-199	1	Spring	2018	A-
76543	CS-101	1	Fall	2017	A
76543	CS-319	2	Spring	2018	A
76653	EE-181	1	Spring	2017	C
98765	CS-101	1	Fall	2017	C-
98765	CS-315	1	Spring	2018	B
98988	BIO-101	1	Summer	2017	A
98988	BIO-301	1	Summer	2018	<i>null</i>

学生 natural join 选课

ID	name	dept_name	tot_cred
00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
19991	Brandt	History	80
23121	Chavez	Finance	110
44553	Peltier	Physics	56
45678	Levy	Physics	46
54321	Williams	Comp. Sci.	54
55739	Sanchez	Music	38
70557	Snow	Physics	0
76543	Brown	Comp. Sci.	58
76653	Aoi	Elec. Eng.	60
98765	Bourikas	Elec. Eng.	98
98988	Tanaka	Biology	120

ID	course_id	sec_id	semester	year	grade
00128	CS-101	1	Fall	2017	A
00128	CS-347	1	Fall	2017	A-
12345	CS-101	1	Fall	2017	C
12345	CS-190	2	Spring	2017	A
12345	CS-315	1	Spring	2018	A
12345	CS-347	1	Fall	2017	A
19991	HIS-351	1	Spring	2018	B
23121	FIN-201	1	Spring	2018	C+
44553	PHY-101	1	Fall	2017	B-
45678	CS-101	1	Fall	2017	F
45678	CS-101	1	Spring	2018	B+
45678	CS-319	1	Spring	2018	B
54321	CS-101	1	Fall	2017	A-
54321	CS-190	2	Spring	2017	B+
55739	MU-199	1	Spring	2018	A-
76543	CS-101	1	Fall	2017	A
76543	CS-319	2	Spring	2018	A
76653	EE-181	1	Spring	2017	C
98765	CS-101	1	Fall	2017	C-
98765	CS-315	1	Spring	2018	B
98988	BIO-101	1	Summer	2017	A
98988	BIO-301	1	Summer	2018	null

ID	name	dept_name	tot_cred	course_id	sec_id	semester	year	grade
00128	Zhang	Comp. Sci.	102	CS-101	1	Fall	2017	A
00128	Zhang	Comp. Sci.	102	CS-347	1	Fall	2017	A-
12345	Shankar	Comp. Sci.	32	CS-101	1	Fall	2017	C
12345	Shankar	Comp. Sci.	32	CS-190	2	Spring	2017	A
12345	Shankar	Comp. Sci.	32	CS-315	1	Spring	2018	A
12345	Shankar	Comp. Sci.	32	CS-347	1	Fall	2017	A
19991	Brandt	History	80	HIS-351	1	Spring	2018	B
23121	Chavez	Finance	110	FIN-201	1	Spring	2018	C+
44553	Peltier	Physics	56	PHY-101	1	Fall	2017	B-
45678	Levy	Physics	46	CS-101	1	Fall	2017	F
45678	Levy	Physics	46	CS-101	1	Spring	2018	B+
45678	Levy	Physics	46	CS-319	1	Spring	2018	B
54321	Williams	Comp. Sci.	54	CS-101	1	Fall	2017	A-
54321	Williams	Comp. Sci.	54	CS-190	2	Spring	2017	B+
55739	Sanchez	Music	38	MU-199	1	Spring	2018	A-
76543	Brown	Comp. Sci.	58	CS-101	1	Fall	2017	A
76543	Brown	Comp. Sci.	58	CS-319	2	Spring	2018	A
76653	Aoi	Elec. Eng.	60	EE-181	1	Spring	2017	C
98765	Bourikas	Elec. Eng.	98	CS-101	1	Fall	2017	C-
98765	Bourikas	Elec. Eng.	98	CS-315	1	Spring	2018	B
98988	Tanaka	Biology	120	BIO-101	1	Summer	2017	A
98988	Tanaka	Biology	120	BIO-301	1	Summer	2018	null

自然连接中的危险情况

- 注意具有**相同名称的不相关属性**，以免被错误地等同起来
- 示例——列出学生、教师的姓名以及他们所修课程的名称
- 正确

```
select name, title  
from student natural join takes, course  
where takes.course_id = course.course_id;
```

- 不正确

```
select name, title  
from student natural join takes natural join course;
```

- 此查询省略了所有学生在其所在院系以外的院系选修课程的对。(学生姓名，课程名称)对。
- 正确的版本（上面）正确输出这样的对。

Takes

Course

course_id	title	dept_name	credits
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

Student

ID	name	dept_name	tot_cred
00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
19991	Brandt	History	80
23121	Chavez	Finance	110
44553	Peltier	Physics	56
45678	Levy	Physics	46
54321	Williams	Comp. Sci.	54
55739	Sanchez	Music	38
70557	Snow	Physics	0
76543	Brown	Comp. Sci.	58
76653	Aoi	Elec. Eng.	60
98765	Bourikas	Elec. Eng.	98
98988	Tanaka	Biology	120

ID	course_id	sec_id	semester	year	grade
00128	CS-101	1	Fall	2017	A
00128	CS-347	1	Fall	2017	A-
12345	CS-101	1	Fall	2017	C
12345	CS-190	2	Spring	2017	A
12345	CS-315	1	Spring	2018	A
12345	CS-347	1	Fall	2017	A
19991	HIS-351	1	Spring	2018	B
23121	FIN-201	1	Spring	2018	C+
44553	PHY-101	1	Fall	2017	B-
45678	CS-101	1	Fall	2017	F
45678	CS-101	1	Spring	2018	B+
45678	CS-319	1	Spring	2018	B
54321	CS-101	1	Fall	2017	A-
54321	CS-190	2	Spring	2017	B+
55739	MU-199	1	Spring	2018	A-
76543	CS-101	1	Fall	2017	A
76543	CS-319	2	Spring	2018	A
76653	EE-181	1	Spring	2017	C
98765	CS-101	1	Fall	2017	C-
98765	CS-315	1	Spring	2018	B
98988	BIO-101	1	Summer	2017	A
98988	BIO-301	1	Summer	2018	NULL

Correct

name	title
Zhang	Intro. to Computer Science
Zhang	Database System Concepts
Shankar	Intro. to Computer Science
Shankar	Game Design
Shankar	Robotics
Shankar	Database System Concepts
Brandt	World History
Chavez	Investment Banking
Peltier	Physical Principles
Levy	Intro. to Computer Science
Levy	Intro. to Computer Science
Levy	Image Processing
Williams	Intro. to Computer Science
Williams	Game Design
Sanchez	Music Video Production
Brown	Intro. to Computer Science
Brown	Image Processing
Aoi	Intro. to Digital Systems
Bourikas	Intro. to Computer Science
Bourikas	Robotics
Tanaka	Intro. to Biology
Tanaka	Genetics

Wrong

name	title
Zhang	Intro. to Computer Science
Zhang	Database System Concepts
Shankar	Intro. to Computer Science
Shankar	Game Design
Shankar	Robotics
Shankar	Database System Concepts
Brandt	World History
Chavez	Investment Banking
Peltier	Physical Principles
Williams	Intro. to Computer Science
Williams	Game Design
Sanchez	Music Video Production
Brown	Intro. to Computer Science
Brown	Image Processing
Aoi	Intro. to Digital Systems
Tanaka	Intro. to Biology
Tanaka	Genetics



自然连接与Using子句的

- 为了避免错误地将属性相等的危险，我们可以使用“ **using** ”结构，该结构允许我们准确指定应该将哪些列相等。
- 查询示例

```
select name, title  
from (student natural join takes) join course using  
(course_id);
```

连接条件

- on条件允许对被连接的关系使用一般谓词
- 这个谓词的写法类似于where子句谓词，除了使用关键字on
- 查询示例

```
select *  
from student join takes on student.ID = takes.ID;
```

- on条件指定，如果来自student的元组与来自take的元组的ID值相等，则它们匹配。
- 相当于：

```
select *  
from student, takes  
where student.ID = takes.ID;
```

外连接 Outer Join

- 避免信息丢失的连接操作的扩展。
- 计算连接，然后将一个关系中与另一个关系中的元组不匹配的元组**添加到连接的结果中**。
- 使用**空值**。
- 外连接的三种形式：
 - 左外连接 **left outer join**
 - 右外连接 **right outer join**
 - 完全外连接 **full outer join**

外连接示例

- 关系 *course*

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>
BIO-301	Genetics	Biology	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3

- 关系 *prereq*

<i>course_id</i>	<i>prereq_id</i>
BIO-301	BIO-101
CS-190	CS-101
CS-347	CS-101

- 观察一下

Course 中 缺少 **CS-347**

Prereq 中 缺少 **CS-315**

左外连接

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>
BIO-301	Genetics	Biology	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3

<i>course_id</i>	<i>prereq_id</i>
BIO-301	BIO-101
CS-190	CS-101
CS-347	CS-101

- **course natural left outer join prereq**

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prereq_id</i>
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-315	Robotics	Comp. Sci.	3	<i>null</i>

- 关系代数: **course $\bowtie$ prereq**

右外连接

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>
BIO-301	Genetics	Biology	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3



<i>course_id</i>	<i>prereq_id</i>
BIO-301	BIO-101
CS-190	CS-101
CS-347	CS-101

- **course natural right outer join prereq**

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prereq_id</i>
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-347	<i>null</i>	<i>null</i>	<i>null</i>	CS-101

- 关系代数: **course $\bowtie$ prereq**

完全外连接

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>
BIO-301	Genetics	Biology	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3



<i>course_id</i>	<i>prereq_id</i>
BIO-301	BIO-101
CS-190	CS-101
CS-347	CS-101

- **course natural full outer join prereq**

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prereq_id</i>
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-315	Robotics	Comp. Sci.	3	<i>null</i>
CS-347	<i>null</i>	<i>null</i>	<i>null</i>	CS-101

- 关系代数: **course $\bowtie$ prereq**

连接类型和条件

- **Join 操作** 采取两个关系并返回另一个关系。
 - 这些附加操作通常用作from子句中的子查询表达式
- **Join 条件** - 定义两个关系中的哪些元组匹配。
- **Join 类型** - 定义如何处理每个关系中与另一个关系中的任何元组不匹配的元组（基于连接条件）。

<i>Join types</i>
inner join
left outer join
right outer join
full outer join

<i>Join conditions</i>
natural
on <predicate>
using ($A_1, A_2, \dots, A_n$)

连接关系 - 示例

- **course natural right outer join prereq**

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prereq_id</i>
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-347	<i>null</i>	<i>null</i>	<i>null</i>	CS-101

- **course full outer join prereq using (course_id)**

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prereq_id</i>
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-315	Robotics	Comp. Sci.	3	<i>null</i>
CS-347	<i>null</i>	<i>null</i>	<i>null</i>	CS-101

连接关系 - 示例

- **course inner join prereq on**
course.course_id = prereq.course_id

course_id	title	dept_name	credits	prereq_id	course_id
BIO-301	Genetics	Biology	4	BIO-101	BIO-301
CS-190	Game Design	Comp. Sci.	4	CS-101	CS-190

- 上述内容与自然连接之间有什么区别?
- **course left outer join prereq on**
course.course_id = prereq.course_id

course_id	title	dept_name	credits	prereq_id	course_id
BIO-301	Genetics	Biology	4	BIO-101	BIO-301
CS-190	Game Design	Comp. Sci.	4	CS-101	CS-190
CS-315	Robotics	Comp. Sci.	3	null	null

连接关系 - 示例

- ***course* natural right outer join *prereq***

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prereq_id</i>
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-347	<i>null</i>	<i>null</i>	<i>null</i>	CS-101

- ***course* full outer join *prereq* using (*course_id*)**

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prereq_id</i>
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-315	Robotics	Comp. Sci.	3	<i>null</i>
CS-347	<i>null</i>	<i>null</i>	<i>null</i>	CS-101

视图

View

视图

- 在某些情况下，不希望所有用户都看到整个逻辑模型（即存储在数据库中的所有实际关系）。
- 假设一个人需要知道导师的姓名和所在部门，但不需要知道薪水。这个人应该看到一个用 SQL 描述的关系，该关系由

```
select ID, name, dept_name  
from instructor;
```

- 视图提供了一种机制，可以隐藏某些用户的视图中的某些数据。
- 任何不属于概念模型但作为“虚拟关系”对用户可见的关系称为视图。

视图的定义

- 视图是使用 **create view** 语句定义的，其形式为

```
create view v as < query expression >
```

其中 < **query expression** > 是任意合法的 **SQL** 表达式。

视图名称用 **v** 表示。

- 视图定义完成后，可以使用视图名称来引用该视图生成的虚拟关系。
- 视图定义不同于通过计算查询表达式来创建新关系。
- 相反，视图定义会**保存表达式**；该表达式会被替换到使用该视图的查询中。

视图定义和使用

- 没有显示工资信息的教员视图

```
create view faculty as  
  select ID, name, dept_name  
  from instructor;
```

- 查找生物系的所有讲师

```
select name  
from faculty  
where dept_name = 'Biology';
```

- 创建系工资总额的视图

```
create view departments_total_salary(dept_name, total_salary) as  
  select dept_name, sum(salary)  
  from instructor  
  group by dept_name;
```

在 SQLite3 数据库中显示视图

- 输入命令:
`.tables`
- 或者
`PRAGMA table_list;`

使用其他视图定义的视图

- 一个视图可以在定义另一个视图的表达式中使用
- 视图关系 v_1 被认为 **直接依赖** 于视图关系 v_2 如果 v_2 被使用在定义 v_1 的表达式中。
- 视图关系 v_1 被认为 **依赖于** 查看关系 v_2 如果 v_1 直接依赖于 v_2 ，或者存在从 v_1 到 v_2 的依赖路径
- 视图关系 v 被称为 **递归的** 如果当前视图依赖于它本身。

使用其他视图定义的视图

```
create view physics_fall_2017 as
  select course.course_id, sec_id, building, room_number
  from course, section
  where course.course_id = section.course_id
        and course.dept_name = 'Physics'
        and section.semester = 'Fall'
        and section.year = '2017';
```

```
create view physics_fall_2017_watson as
  select course_id, room_number
  from physics_fall_2017
  where building= 'Watson';
```

视图扩展

- 展开一个视图

```
create view physics_fall_2017_watson as
  select course_id, room_number
  from physics_fall_2017
  where building= 'Watson';
```

- 成:

```
create view physics_fall_2017_watson as
  select course_id, room_number
  from (select course.course_id, building, room_number
        from course, section
        where course.course_id = section.course_id
          and course.dept_name = 'Physics'
          and section.semester = 'Fall'
          and section.year = '2017')
  where building= 'Watson';
```

视图扩展 (续)

- 一种根据其他视图来定义视图含义的方法。
- 让视图 v_1 由表达式 e_1 定义，该表达式本身可能包含视图关系的用途。
- 表达式的视图扩展重复以下替换步骤：
 - 重复
 - 在 e_1 中查找任何视图关系
 - 用定义 v_i 的表达式替换视图关系 v_i
 - 直到 e_1 中没有更多的视图关系
- 只要视图定义不是递归的，这个循环就会终止

物化视图

- 某些数据库系统允许视图关系以**物理方式存储**。
 - 定义视图时创建的物理副本。
 - 这样的视图称为物化视图
- 如果查询中使用的关系被更新，则物化视图结果将**变得过时**
 - 需要维护视图，即每当底层关系更新时更新视图。
 - 现代数据库系统具有一些内置的视图维护能力。

视图的更新

- 向我们之前定义的教师视图添加一个新的元组

```
insert into faculty  
values ('30765', 'Green', 'Music');
```

- 这种插入必须通过插入到 **教员** 关系中表示
 - 必须具有一个值给予 **工资** 属性。
- 两种方法
 - 拒绝插入
 - 插入元组

('30765', 'Green', 'Music', null)
into the instructor relation

某些更新无法被唯一的解释

```
create view instructor_info as  
  select ID, name, building  
  from instructor, department  
  where instructor.dept_name = department.dept_name;
```

```
insert into instructor_info  
  values ('69987', 'White', 'Taylor');
```

- 问题
 - 如果 Taylor 有多个部门，那么是哪个部门？
 - 如果 Taylor 没有部门怎么办？

有些则完全没有

```
create view history_instructors as  
  select *  
  from instructor  
  where dept_name = 'History';
```

- 会出现什么情况？如果我们插入
('25566', 'Brown', 'Biology', 100000)
进入 *history_instructors* ?

在 SQL 中视图更新的条件

- 大多数 SQL 实现仅允许对简单视图进行更新
 - from 子句只有一个数据库关系。
 - select 子句仅包含关系的属性名称，并且不包含任何表达式、聚合或 distinct 特点。
 - select 子句中未列出的属性都可以设置为 null
 - 查询没有 group by 或 having 子句。
- 通常更好的方式是：使用触发机制（**trigger mechanism**）。

事务

- **事务**由一系列查询和/或更新语句组成，是一个工作“单元”
- **SQL** 标准指定在执行 **SQL** 语句时隐式开始事务。
- 事务必须以下列语句之一结束：
 - **提交工作 (Commit work)**: 事务执行的更新在数据库中成为永久的。
 - **回滚工作 (Rollback work)**: 事务中的SQL语句执行的所有更新均被撤消。
- **原子事务 (Atomic transaction)**
 - 要么完全执行，要么回滚，就像从未发生过一样
- 与并发事务**隔离**

完整性约束 Integrity Constraints

- 完整性约束通过确保对数据库的授权更改不会导致数据一致性的丧失，防止对数据库的意外损坏。
 - 支票账户余额必须大于 10,000.00 美元
 - 银行员工的工资必须至少为每小时 4.00 美元
 - 客户必须有一个（非空）电话号码

单一关系上的约束

- 非空 **not null**
- 主键 **primary key**
- 唯一 **unique**
- **check (P)**, 其中 **P** 是谓词

非空约束

- **not null**
 - 声明名称和预算不为空

name	varchar(20) not null
budget	numeric(12,2) not null

唯一约束

- **unique** ($A_1, A_2, \dots, A_m$)
 - **unique**规范指出属性 $A_1, A_2, \dots, A_m$ 形成候选键。
 - 候选键可以为空（与主键相反）。

检查条款 **check**

- **Check(P)**子句指定一个谓词 P，关系中的每个元组都必须满足该谓词。
 - 例如：确保学期是秋季、冬季、春季或夏季之一

```
create table section(  
    course_id      varchar (8),  
    sec_id         varchar (8),  
    semester       varchar (6),  
    year           numeric (4,0),  
    building       varchar (15),  
    room_number    varchar (7),  
    time slot id   varchar (4),  
    primary key (course_id, sec_id, semester, year),  
    check (semester in ('Fall', 'Winter', 'Spring', 'Summer'))  
);
```

参照完整性 Referential Integrity

- 确保在一个关系中针对给定的一组属性出现的值也出现在另一个关系中的某组属性中。
 - 例如：如果“生物学”是出现在讲师关系中的一个元组中的系名称，那么系关系中就存在一个“生物学”的元组。
- 设 A 为一组属性。
- 设 R 和 S 是两个包含属性 A 的关系，其中 A 是 S 的主键。
- 如果 A 中出现在 R 中的任何值也出现在 S 中，则称 A 是 R 的**外键**。

参照完整性 (续)

- 外键可以作为 SQL 创建表声明的一部分

```
foreign key (dept_name) references department
```

- 默认情况下，外键引用被引用表的主键属性。
- SQL 允许明确指定引用关系的属性列表。

```
foreign key (dept_name) references department (dept_name)
```

参照完整性中的级联操作

Cascading Actions

- 当违反参照完整性约束时，正常程序是拒绝导致违反的操作。
- 如果要删除或更新，另一种方法是级联

```
create table course (  
...  
    dept_name varchar(20),  
    foreign key (dept_name) references department  
        on delete cascade  
        on update cascade,  
...  
);
```

- 我们可以使用以下方法来代替级联：
 - set null ,
 - set default

事务期间违反完整性约束

- 考虑:

```
create table person (  
    ID          char(10),  
    name        char(40),  
    mother      char(10),  
    father      char(10),  
    primary key (ID),  
    foreign key (father) references person,  
    foreign key (mother) references person  
);
```

- 如何插入元组而不导致违反约束?
 - 在插入 person 之前插入一个人的father和mother
 - 或者, 最初将father和mother设置为空, 在插入所有人后更新 (如果父亲和母亲属性声明为非空则不可能)
 - 或推迟约束检查

复杂检查条件

- 检查子句中的谓词可以是任意谓词，可以包含子查询。

```
check (time_slot_id in (select time_slot_id from time_slot))
```

检查条件表明section关系中每个元组中的time_slot_id实际上是time_slot关系中一个时间段的标识符。

- 不仅在部分中插入或修改元组时需要检查条件，而且在关系time_slot发生变化时也需要检查条件

断言 Assertions

- 断言 是一个谓词，表达我们希望数据库始终满足的条件。
- 以下约束可以用断言来表达：
 - 学生关系中的每个元组，属性 *tot_cred* 的值必须等于学生已成功完成的课程学分总和。
 - 一个教师不能在一个学期的同一时间段在两个不同的教室授课
- SQL 中的断言采用以下形式：

```
create assertion <assertion-name> check (<predicate>);
```

SQL 中的内置数据类型

- **date** : 日期, 包含 (4 位数字) 年份、月份和日期
 - 例如: 日期 '2005-7-27'
- **time** : 一天中的时间, 以小时、分钟和秒为单位。
 - 例如: 时间 '09:00:30' 时间 '09:00:30.75'
- **Timestamp (时间戳)**: 日期加时间
 - 例如: 时间戳 '2005-7-27 09:00:30.75'
- **Interval (间隔)**: 一段时间
 - 例如: 间隔 '1' 天
 - 从另一个日期/时间/时间戳值中减去一个日期/时间/时间戳值, 得到一个间隔值
 - 可以将间隔值添加到日期/时间/时间戳值

SQLite 支持的7中日期和时间方法

- **date**(*time-value*, *modifier*, *modifier*, ...)
- **time**(*time-value*, *modifier*, *modifier*, ...)
- **datetime**(*time-value*, *modifier*, *modifier*, ...)
- **julianday**(*time-value*, *modifier*, *modifier*, ...)
 - since noon in Greenwich on November 24, 4714 B.C.
- **unixepoch**(*time-value*, *modifier*, *modifier*, ...)
 - a unix timestamp
- **strftime**(*format*, *time-value*, *modifier*, *modifier*, ...)
 - date formatted according to the format string
- **timediff**(*time-value*, *time-value*)
 - 示例: SELECT timediff('2023-02-15','2023-03-15');

https://sqlite.org/lang_datefunc.html

大对象类型 Large-Object Types

- 大型对象（照片、视频、CAD 文件等）存储为大对象：
 - **blob**：二进制大对象——对象是未解释的二进制数据的大型集合（其解释留给数据库系统之外的应用程序）
 - **clob**：字符大对象——对象是字符数据的大型集合
- 当查询返回大对象时，将返回一个指针而不是大对象本身。

用户定义类型

- SQL 中的创建类型构造创建用户定义类型

```
create type Dollars as numeric (12,2);
```

- 例子:

```
create table department(  
    dept_name    varchar (20),  
    building     varchar (15),  
    budget       Dollars  
);
```

域 Domains

- 在 SQL-92 中创建域构造创建用户定义的域类型

```
create domain person_name char(20) not null
```

- 类型和域类似。域可以指定约束，例如非空。
- 例子：

```
create domain degree_level varchar(10)  
    constraint degree_level_test  
        check (value in ('Bachelors', 'Masters', 'Doctorate'));
```

索引|创建 **Index**

- 许多查询仅引用表中的一小部分记录。
- 系统读取每条记录来查找具有特定值的记录效率低下
- 关系属性上的索引是一种数据结构，它允许数据库系统有效地找到关系中具有该属性指定值的元组，而无需扫描关系的所有元组。
- **create index**命令创建索引

```
create index <name> on <relation-name> (attribute);
```

索引创建示例

- ```
create table student (
 ID varchar (5),
 name varchar (20) not null,
 dept_name varchar (20),
 tot_cred numeric (3,0) default 0,
 primary key (ID)
);
```

- ```
create index studentID_index on student(ID);
```

- 查询:

```
select * from student where ID = '12345';
```

可以通过使用索引来查找所需的记录，而不需要查看学生的所有记录

授权 Authorization

- 我们可能会为用户分配数据库各个部分的几种形式的授权。
 - select- 允许读取，但不允许修改数据。
 - insert- 允许插入新数据，但不允许修改现有数据。
 - update- 允许修改，但不允许删除数据。
 - delete- 允许删除数据。
- 每种**授权**类型都称为一种**权限**。
- 我们可以授予用户对数据库指定部分（例如关系或视图）的全部权限、无权限或这些权限的组合。

授权 (续)

- 修改 数据库模式 的授权形式
 - 索引— 允许创建和删除索引。
 - 资源— 允许创建新的关系。
 - 更改— 允许添加或删除关系中的属性。
 - 删除— 允许删除关系。

SQL 中的授权规范

- **grant** 语句用于授予授权

```
grant <privilege list> on <relation or view > to <user list>;
```

- **<user list>**是：
 - 用户 ID
 - **public** , 允许所有有效用户授予特权
 - 角色, **role** (稍后详细介绍)
- 例子:

```
grant select on department to Amit, Satoshi;
```

- 授予视图上的权限并不意味着授予底层关系上的任何权限。
- 权限的授予者必须已经拥有指定项目的权限 (或者是数据库管理员) 。

SQL 中的权限

- **select** : 允许对关系进行读取访问，或者使用视图进行查询的能力
 - 示例：授予用户 U1、U2 和 U3 对讲师关系的选择授权：

```
grant select on instructor to U1, U2, U3;
```
- **insert** : 插入元组的能力
- **update** : 使用 SQL 更新语句进行更新的能力
- **delete** : 删除元组的能力。
- **all privileges** : 用作所有允许权限的缩写形式

在 SQL 中撤销授权 Revoking

- **revoke** 语句用于撤销授权。

```
revoke <privilege list> on <relation or view> from <user list>;
```

- 例子:

```
revoke select on student from U1, U2, U3;
```

- **<privilege-list>** 可以是all，表示撤销被撤销者可能拥有的所有特权。
- 如果 **<revokee -list>** 包含public，则除明确授予权限的用户外，所有用户都会失去权限。
- 如果不同的被授予者向同一用户授予了两次相同的权限，则用户可以在撤销后保留该权限。
- 所有依赖于被撤销特权的特权也将被撤销。

角色 Roles

- 角色是一种区分不同用户的方式，根据这些用户可以在数据库中访问/更新什么。
- 要创建角色，我们使用：

```
create role <name>;
```

- 例子：

```
create role instructor;
```

- 创建角色后，我们可以使用以下方式将“用户”分配给该角色：

```
grant <role> to <users>;
```

角色示例

```
create role instructor;  
grant instructor to Amit;
```

- 可以授予角色特权:

```
grant select on takes to instructor;
```

- 可以将角色授予用户，也可以授予其他角色

```
create role teaching_assistant;  
grant teaching_assistant to instructor;
```

- Instructor inherits all privileges of teaching_assistant

- 角色链

```
create role dean;  
grant instructor to dean;  
grant dean to Satoshi;
```

视图授权

```
create view geo_instructor as  
  (select *  
   from instructor  
   where dept_name = 'Geology');  
  
grant select on geo_instructor to geo_staff;
```

假设geo_staff成员发出

```
select *  
from geo_instructor;
```

- 如果
 - geo_staff 没有对 instructor 的权限?
 - 视图的创建者对 instructor 没有某些权限?

其他授权功能

- 引用创建外键的权限

```
grant reference (dept_name) on department to Mariano;
```

- 为什么需要这样做?

- 特权的转移

```
grant select on department to Amit with grant option;  
revoke select on department from Amit, Satoshi cascade;  
revoke select on department from Amit, Satoshi restrict;
```

数据库系统 Database System 谢谢!

问/答?

Instructor: Yiwen Shen (沈轶闻), Ph.D.
Dept. of Software & Computer Engineering,
Ajou University, Korea
chrisshen@ajou.ac.kr